

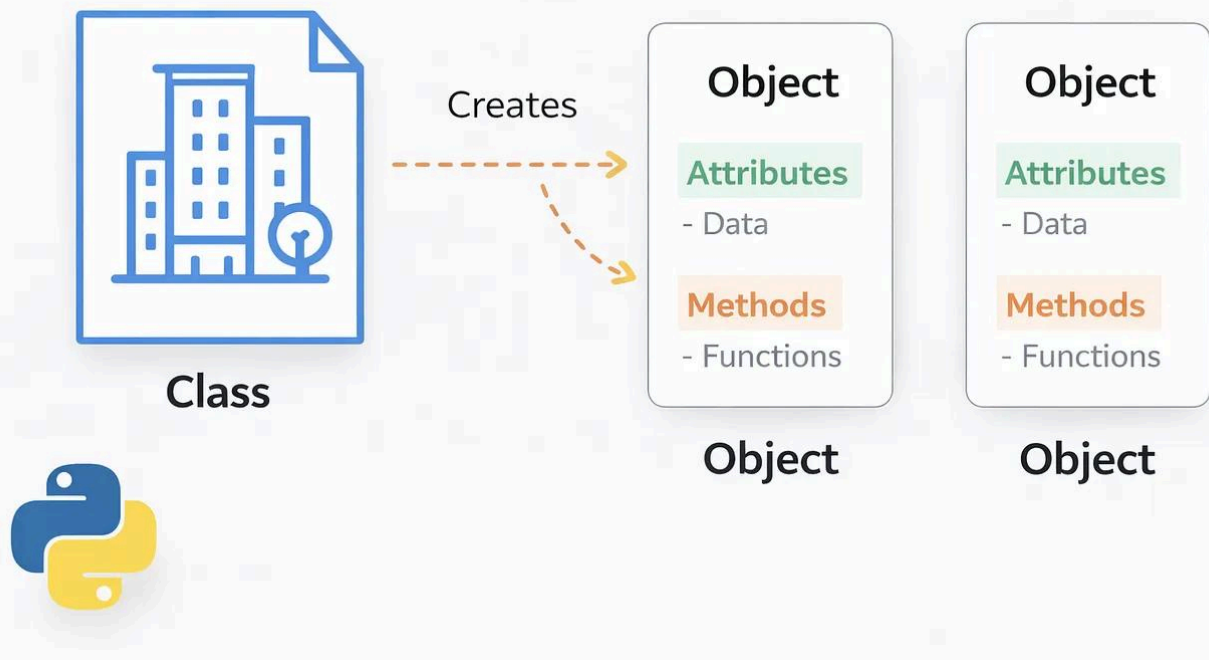
PART 5 — OOP (Object-Oriented Programming)

OOP becomes much easier if you stop thinking of it as **17 separate topics**.

Think:

**Class = blueprint → Object = actual thing → Methods = behavior → Variables = state →
Inheritance = reuse → MRO/super = control inheritance behavior → Abstract
classes/Protocols = contracts → Dunder methods = Python integration.**

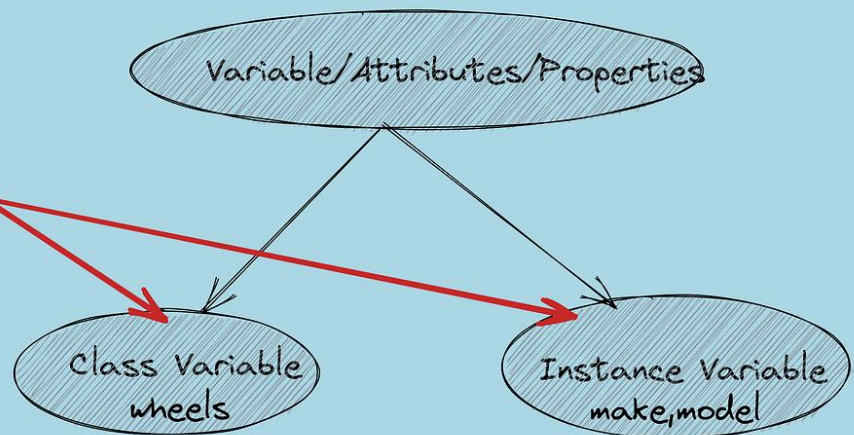
Object Oriented Programming (OOP)



```
class Car:
    # Class variable
    wheels = 4

    def __init__(self, make, model):
        # Object variables
        self.make = make
        self.model = model

car1 = Car("Honda", "Civic")
car2 = Car("Toyota", "Corolla")
```

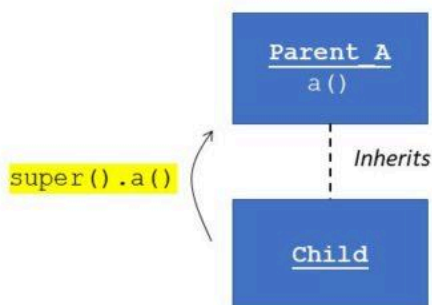


Python super ()

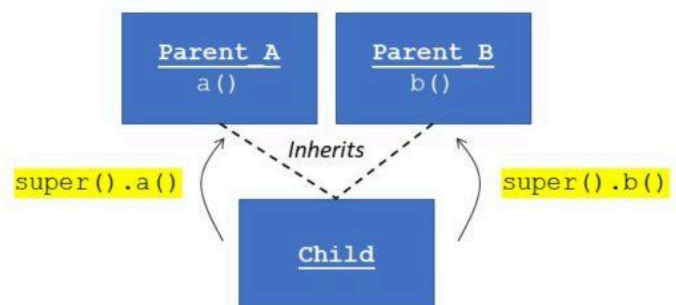


Returns temporary object of the superclass to help you access its methods. Its purpose is to avoid using the base class name explicitly. It also enables **multiple inheritance**.

A) Single Inheritance



B) Multiple Inheritance



finxter

Abstraction in Python

Abstract Class: Shape

- Abstract Method: `area()`
Must be implemented by subclasses
- Abstract Method: `perimeter()`
Must be implemented by subclasses

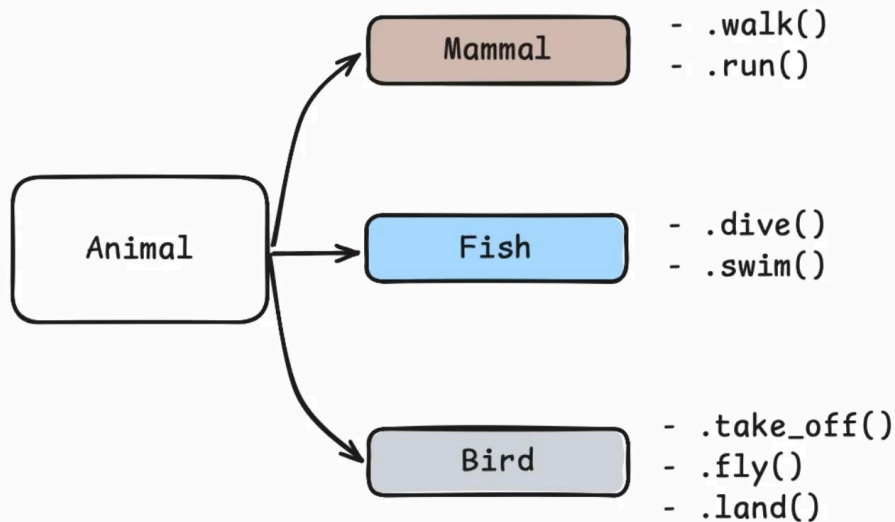
Subclass: Circle

- Implements `area()` and `perimeter()` for circle

Subclass: Rectangle

- Implements `area()` and `perimeter()` for rectangle

Understanding Protocols



```

1 class base1():
2     def __init__(self):
3         print("In the base1 class")
4         self.s1= "base1 class var"
5 class base2():
6     def __init__(self):
7         print("In the base2 class")
8         self.s2= "base2 class var"
9
10 class derived(base1,base2):
11     def __init__(self):
12         print("In the derived class")
13         super().__init__()           # call base1 __init__
14         super(base1,self).__init__()  # call base2 __init__
15 print(derived.__mro__)

```

input

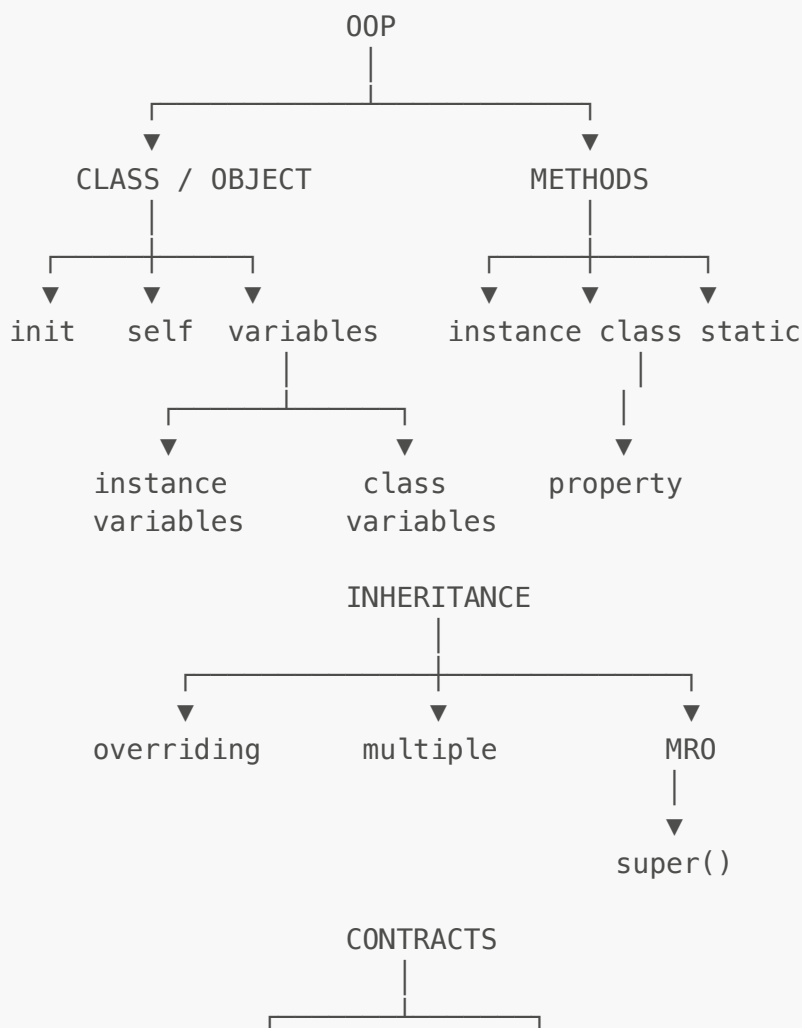
```

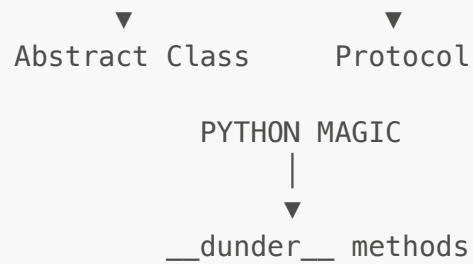
(<class '__main__.derived'>, <class '__main__.base1'>, <class '__main__.base2'>, <class 'object'>)

```



THE BIG OOP MAP





1. Class & Object

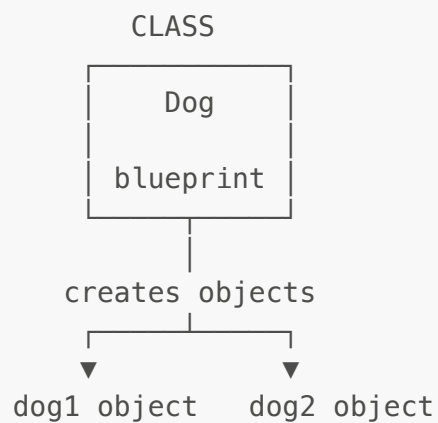
Class = blueprint

```
class Dog:
    pass
```

Object = instance created from the class

```
dog1 = Dog()
dog2 = Dog()
```

Visual:



Think:

```
Class = blueprint
Object = actual thing
```

Like:

```
Class → Car blueprint  
Object → my_car  
Object → your_car
```

2. `__init__`

`__init__` initializes a newly created object's state.

```
class Dog:  
  
    def __init__(self, name, age):  
        self.name = name  
        self.age = age
```

Now:

```
dog = Dog("Tom", 3)
```

Conceptually:

```
Dog("Tom", 3)  
  |  
  ▼  
create object  
  |  
  ▼  
__init__(self, "Tom", 3)  
  |  
  ▼  
self.name = "Tom"  
self.age = 3
```

⚠ Important

Strictly speaking, `__init__` **initializes** the object; object creation itself is associated with `__new__`.

For normal Python OOP, however, you usually focus on:

```
object created  
  ↓
```

```
__init__ initializes it
```

3. self

self means:

the current object/instance

```
class Dog:

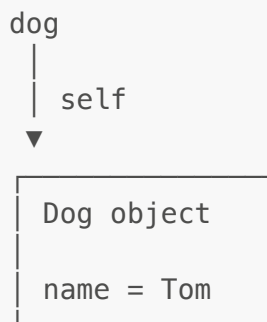
    def set_name(self, name):
        self.name = name

    def show(self):
        print(self.name)
```

```
dog = Dog()

dog.set_name("Tom")
dog.show()
```

Visual:



When you write:

```
dog.show()
```

Python conceptually passes the object:

```
Dog.show(dog)
```

That's why the method has:

```
self
```

as its first parameter.

4. Instance Variables

These belong to **one particular object**.

```
class Car:
    def __init__(self, brand, color):
        self.brand = brand
        self.color = color
```

```
car1 = Car("Toyota", "Red")
car2 = Car("Honda", "Blue")
```

Visual:



Each object has its own state.

Mental model

```
self.x → instance variable
```

5. Class Variables

Class variables belong to the **class** and are generally shared.

```
class Car:  
    wheels = 4
```

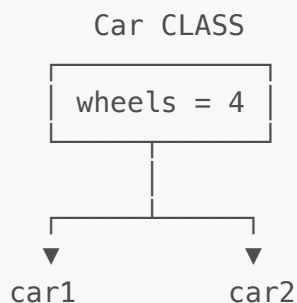
Now:

```
car1 = Car()  
car2 = Car()
```

Both can access:

```
car1.wheels  
car2.wheels
```

Visual:



Difference

```
Instance variable  
  ↓  
belongs to each object  
  
Class variable  
  ↓  
belongs to the class / shared class-level state
```

6. Instance Methods

Normal methods operating on an object's data.

```
class Car:

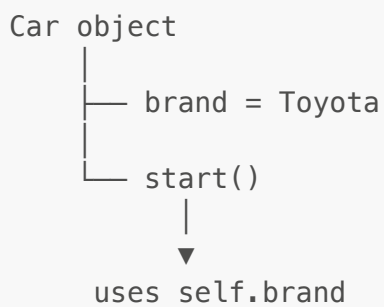
    def __init__(self, brand):
        self.brand = brand

    def start(self):
        return f"{self.brand} started"
```

```
car = Car("Toyota")

car.start()
```

Visual:



The diagram shows a 'Car object' with two attributes: 'brand = Toyota' and 'start()'. An arrow points from 'start()' to the text 'uses self.brand', indicating that the method uses the 'brand' attribute of the object.

First parameter:

```
self
```

7. @classmethod

A class method receives the **class** as its first argument.

Convention:

```
cls
```

Example:

```
class Car:

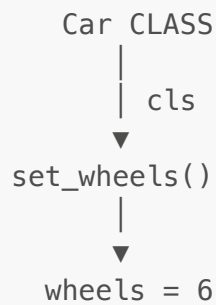
    wheels = 4

    @classmethod
    def set_wheels(cls, number):
        cls.wheels = number
```

Call:

```
Car.set_wheels(6)
```

Visual:



Common use

Alternative constructors:

```
class User:

    def __init__(self, name):
        self.name = name

    @classmethod
    def from_string(cls, text):
        return cls(text.strip())
```

Then:

```
user = User.from_string(" Alice ")
```

Remember

```
instance method → self  
class method   → cls
```

8. @staticmethod

A static method doesn't automatically receive:

```
self
```

or:

```
cls
```

Example:

```
class Math:  
  
    @staticmethod  
    def add(a, b):  
        return a + b
```

Call:

```
Math.add(2, 3)
```

It is basically a function logically grouped inside the class.

Mental model

```
instance method → needs object  
classmethod     → needs class  
staticmethod    → needs neither automatically
```

9. @property

This lets you expose method logic like an attribute.

Instead of:

```
circle.get_radius()
```

you can write:

```
circle.radius
```

Example:

```
class Circle:

    def __init__(self, radius):
        self._radius = radius

    @property
    def radius(self):
        return self._radius
```

Now:

```
c = Circle(5)

print(c.radius)
```

Visual:

```
c.radius
  |
  ▼
@property
  |
  ▼
radius()
  |
  ▼
self._radius
```

You can also provide a setter:

```
@radius.setter
def radius(self, value):
    if value < 0:
        raise ValueError("Invalid radius")
    self._radius = value
```

Now:

```
c.radius = 10
```

can perform validation.

Mental model

property gives you **attribute syntax with method behavior**.

10. Inheritance

Inheritance means:

A child class gets behavior from a parent class.

```
class Animal:

    def speak(self):
        print("Some sound")

class Dog(Animal):
    pass
```

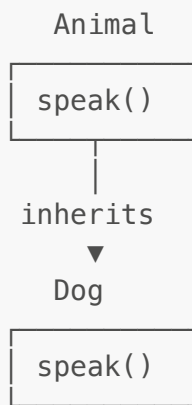
Now:

```
dog = Dog()

dog.speak()
```

works.

Visual:



Mental model

```
Parent
  ↓
Child
  ↓
inherits behavior
```

11. Method Overriding

A child can replace a parent's implementation.

```
class Animal:
    def speak(self):
        return "Some sound"

class Dog(Animal):
    def speak(self):
        return "Bark"
```

Now:

```
dog = Dog()

print(dog.speak())
```

Output:

```
Bark
```

Visual:

```
Animal.speak()  
  |  
  | overridden  
  ▼  
Dog.speak()  
  |  
  ▼  
"Bark"
```

This is **polymorphic behavior**.

12. Multiple Inheritance

A class can inherit from more than one parent.

```
class A:  
    def hello(self):  
        print("A")  
  
class B:  
    def hello(self):  
        print("B")  
  
class C(A, B):  
    pass
```

Now:

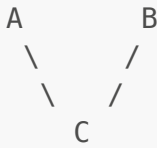
```
c = C()  
  
c.hello()
```

Which `hello()`?

Python needs an order.

That's where **MRO** comes in.

Visual:



13. MRO

MRO = **Method Resolution Order**.

It answers:

When I call a method, where does Python look first?

For:

```
class C(A, B):  
    pass
```

you can inspect:

```
C.mro()
```

Conceptually:

```
C  
↓  
A  
↓  
B  
↓  
object
```

Python searches in that order.

For more complex inheritance, Python uses the **C3 linearization** algorithm to construct the MRO.

🔥 Remember

```
MRO = "Where will Python search?"
```

14. `super()`

`super()` lets you access behavior from the next class in the MRO.

Example:

```
class Animal:
    def speak(self):
        return "Some sound"

class Dog(Animal):
    def speak(self):
        parent = super().speak()
        return parent + " + Bark"
```

Then:

```
dog.speak()
```

Result:

```
Some sound + Bark
```

Visual:

```
Dog.speak()
  |
  ▼
super().speak()
  |
  ▼
Animal.speak()
  |
  ▼
"Some sound"
  |
```

▼
Dog adds " + Bark"

Very important

Don't think:

`super()` always means "my parent".

Better:

`super()` follows the MRO to access the next implementation.

This distinction becomes important with multiple inheritance.

15. Abstract Classes

An abstract class defines a **contract** for subclasses.

Python provides this through `abc`.

```
from abc import ABC, abstractmethod

class Shape(ABC):

    @abstractmethod
    def area(self):
        pass
```

Now:

```
class Circle(Shape):

    def __init__(self, radius):
        self.radius = radius

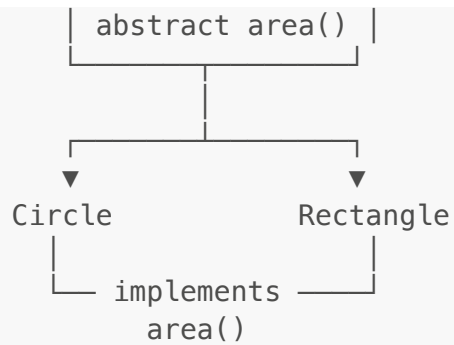
    def area(self):
        return 3.14 * self.radius ** 2
```

You cannot normally instantiate `Shape` directly because it has an unimplemented abstract method.

Visual:

Shape





Mental model

Abstract class = "You MUST provide this behavior."

16. Protocols

Protocols provide **structural typing**.

This is a very powerful modern Python concept.

Instead of asking:

"Does this class inherit from `Shape`?"

a Protocol can say:

"Does this object have the required behavior?"

Example:

```
from typing import Protocol

class HasArea(Protocol):

    def area(self) -> float:
        ...
```

Now:

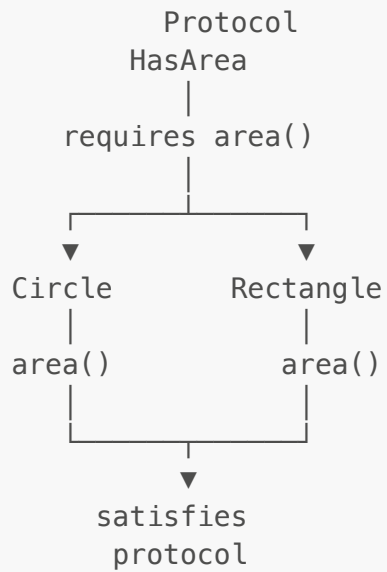
```
class Circle:

    def area(self) -> float:
        return 10.5
```

`Circle` doesn't have to inherit from `HasArea`.

It simply satisfies the structure.

Visual:



Abstract class vs Protocol

```

ABC
├── "You belong to my inheritance hierarchy."

Protocol
├── "You behave according to my interface."
  
```

This is essentially:

```

ABC      → nominal / inheritance-oriented
Protocol → structural / behavior-oriented
  
```

17. Magic / Dunder Methods

"Dunder" means:

double underscore

Examples:

```
__init__  
__str__  
__repr__  
__len__  
__eq__  
__add__  
__call__
```

These allow your objects to interact naturally with Python.

__str__

Controls human-readable representation.

```
class User:  
  
    def __init__(self, name):  
        self.name = name  
  
    def __str__(self):  
        return self.name
```

Then:

```
print(User("Alice"))
```

Python calls:

```
__str__()
```

__len__

Allows:

```
len(obj)
```

to work.

__eq__

a == b

$$a + b$$

obj()

```
class Greeter:

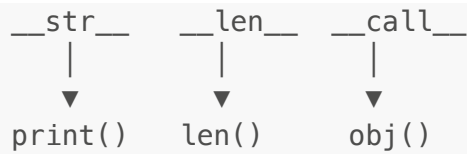
    def __call__(self, name):
        return f"Hello {name}"

g = Greeter()

print(g("Alice"))
```

```

graph TD
    OBJECT --> H1[ ]
    H1 --> T1[ ]
    H1 --> T2[ ]
    H1 --> T3[ ]
    style H1 width:0px,height:0px
    style T1 width:0px,height:0px
    style T2 width:0px,height:0px
    style T3 width:0px,height:0px
  
```

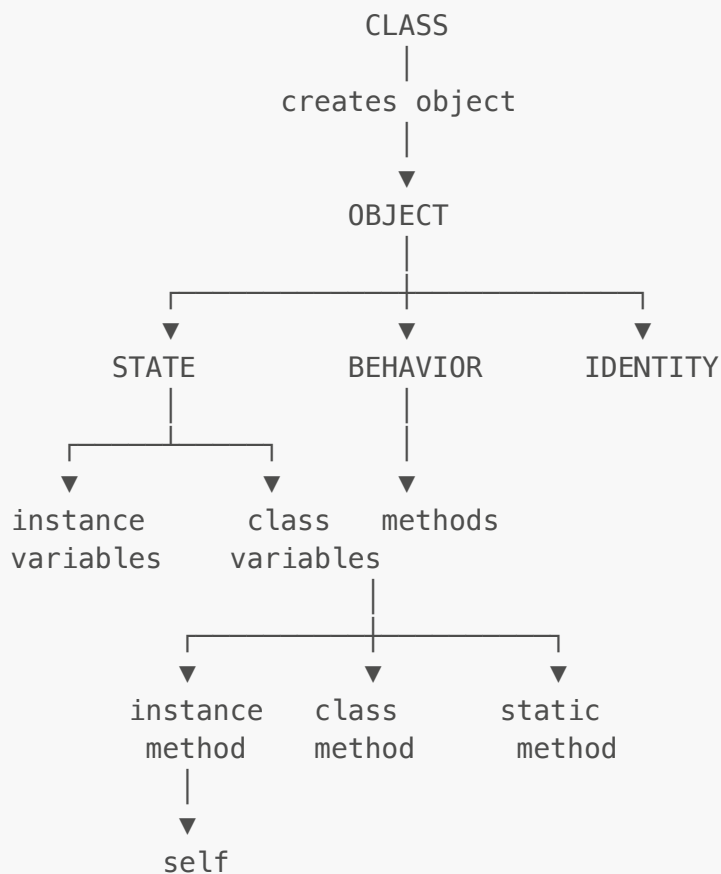


🔥 Mental model

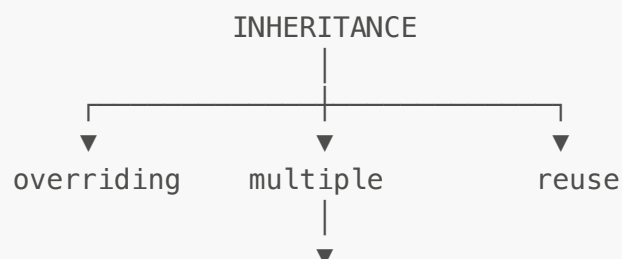
Dunder methods are Python's **protocol hooks** that let your objects participate in built-in syntax and operations.

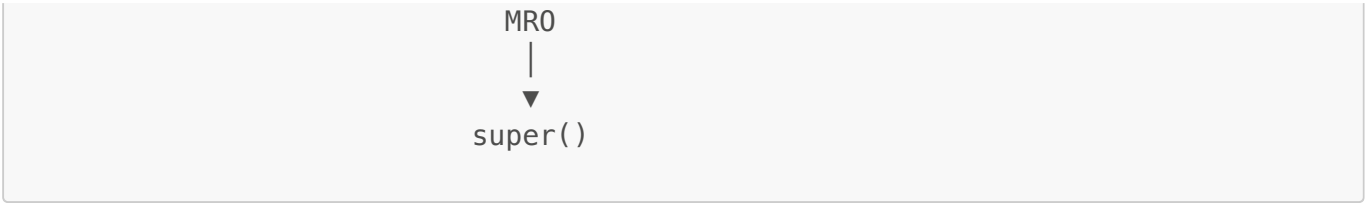
🧠 THE COMPLETE OOP CONNECTION

Now connect everything:

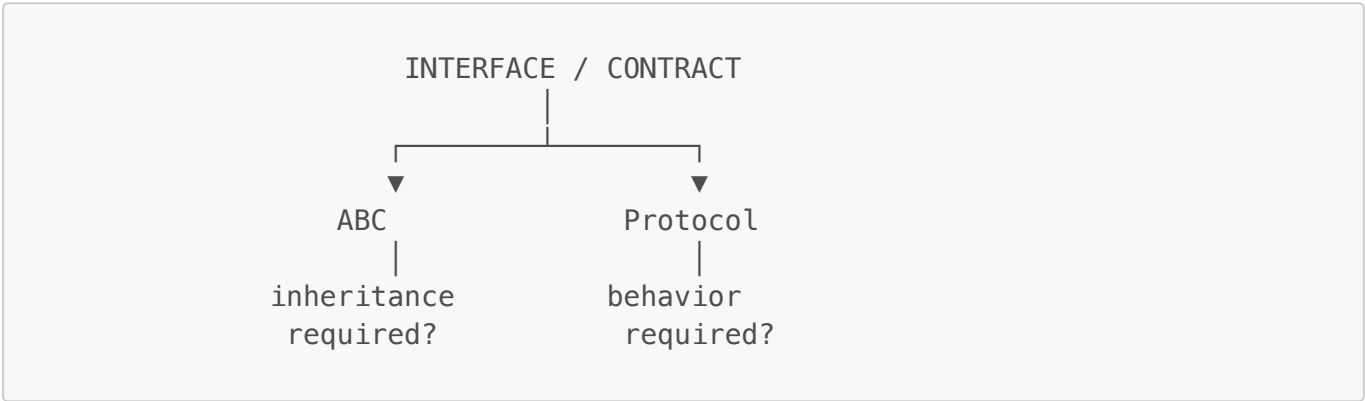


Then:

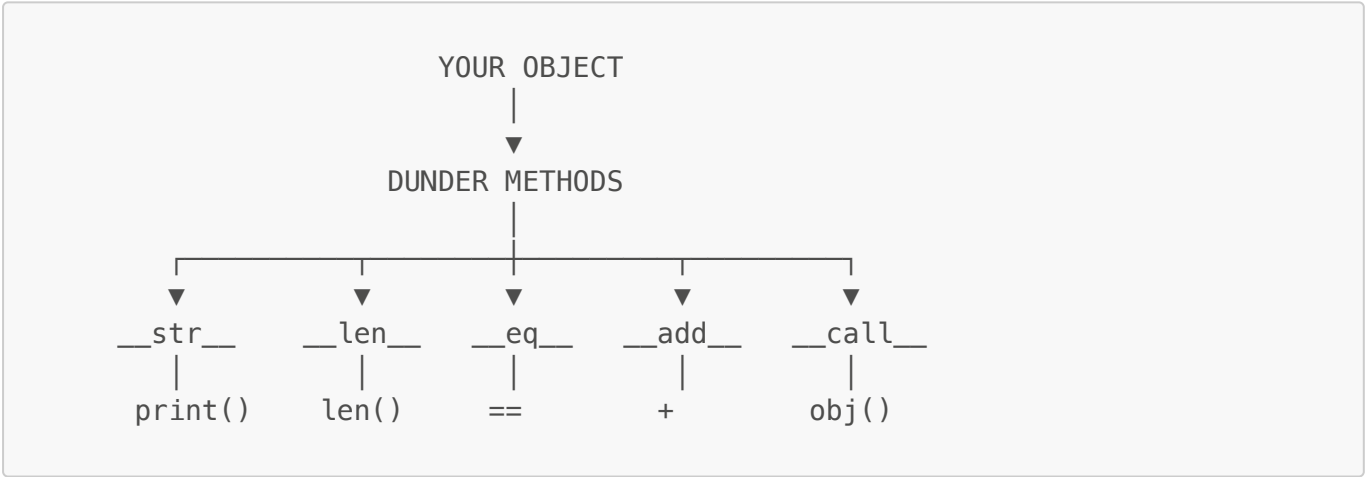




And contracts:



And Python integration:



 The OOP cheat sheet

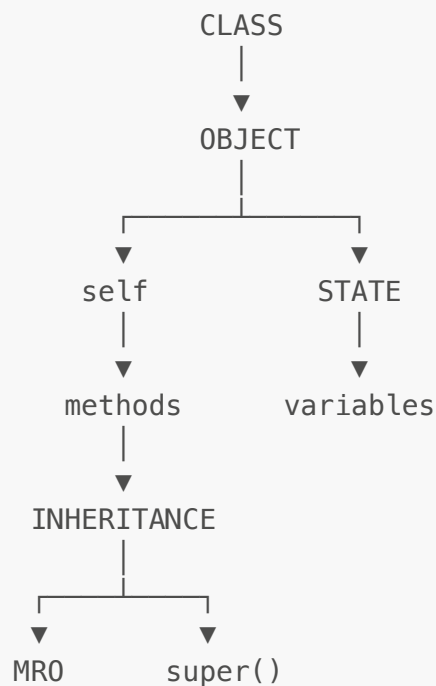
Concept	Mental model
Class	Blueprint
Object	Instance
<code>__init__</code>	Initialize object
<code>self</code>	Current instance
Instance variable	Per-object state
Class variable	Class-level/shared state
Instance method	Works with instance

Concept	Mental model
<code>@classmethod</code>	Works with class
<code>@staticmethod</code>	Independent utility grouped in class
<code>@property</code>	Attribute syntax + method logic
Inheritance	Reuse parent behavior
Overriding	Child replaces behavior
Multiple inheritance	Multiple parents
MRO	Search order for methods
<code>super()</code>	Next implementation in MRO
Abstract class	Enforced interface/contract
Protocol	Structural interface
Dunder methods	Python behavior hooks

The 7 concepts to MASTER first

Don't give all 17 equal weight.

Focus heavily on:



Then add:

```
ABC
|
▼
Protocol
|
▼
Dunder methods
```

★ One sentence for each

Class	→ blueprint
Object	→ actual instance
self	→ current object
__init__	→ initialize state
Inheritance	→ reuse behavior
MRO	→ decide where method is found
super()	→ continue along MRO
ABC	→ enforce implementation
Protocol	→ require behavior, not inheritance
Dunder	→ connect objects to Python syntax

The deepest OOP idea is this: don't memorize `self`, `super()`, MRO, `classmethod`, etc. independently. Understand that **Python objects have state + behavior, classes organize that state/behavior, inheritance creates relationships, MRO determines lookup, and dunder methods define how your objects participate in Python's language protocols.** Once that picture is clear, the individual features become much easier.